Welcome to GVE Interface's documentation!

This module implements the Global Scripter to GVE interface protocol. The module manages the connections to send and receive commands to and from the GVE Server, and maintains a heartbeat to maintain the connection to the GVE Server.

The module only provides an interface to GVE; the actual functionality must be written by the GS programmer.

Controllers running GS and its devices are added to the GVE Help Desk via the "Extron Programmable" tab in GVE's Controller Manager.

Device Specifications

Device Type:	Asset Management Software
Manufacturer:	Extron
Software Version:	2.9
Model(s):	GVE Server

IP Link Pro xi Series

Product	Firmware	ControlScript Pro xi
IPCP Pro xi Firmware	1.03.0000-b008	1.2.4
IPCP Pro Q xi Firmware	1.02.0000-b009	1.2.4
TouchLink Pro 1025/725/525 with LinkLicense	3.06.0003-b002	1.1.7
TouchLink Pro 300M with LinkLicense	1.01.0002-b005	1.1.7

IP Link Pro Series

Product	Firmware	ControlScript Pro
IPL Pro Firmware	3.15.0000-b004	3.13.11

Software

Software Product	Version
GS Version	2.14.0

Version History

Version	Date	Description
2.2.0	2/14/22	Added a parameter to specify the listening interface (e.g. 'LAN', 'AVLAN').
		Reverted previously fixed issue with UDP Sender when using hostnames where the IP Address of GVE server may change. The UDP Sender will no longer be

2.1.1	10/6/20	recreated – the error will be reported. Fixed issue when Secondary Processor is unavailable on initial startup.
2.1.0	7/11/19	Fixed issue with UDP Sender when using hostnames where the IP Address of GVE server may change. Using Timer for UDP heartbeat. Fixed escape character handling.
2.0.0	1/19/18	Changed device IDs to be string instead of integer. IDs can now be any alphanumeric string up to 50 characters.
1.2.2	10/28/17	Fixed typos in comments.
1.2.1	9/8/17	Changed Cool/Warm to Cooling/Warming.
1.2.0	8/18/17	Added support for Room Events.
1.1.0	9/28/16	Updated to use EthernetServerInterfaceEx and Processor.MACAddress.
1.0.1	12/15/15	Initial Extron Certified Version.

Module

Note:

- This module communicates via both TCP and UDP on Port 5555.
- This module sends 'heart beat' messages every 20 seconds via UDP.
- The GVE Server will not begin communication until the first heatbeat message is received.

class **gveClient**(*Hostname*, *HostDevice*, *IPPort=5555*, *Interface='Any'*)

GVE Client Interface Class

This module implements the GS to GVE interface protocol. The module manages the connections to send and receive commands to and from the GVE Server, and maintains a heartbeat to maintain the connection to the GVE Server.

Parameters:

- **Hostname** (*string*) – GVE Server Host Name/IP Address
- **HostDevice** (*extronlib.device.ProcessorDevice*) – Primary Processor
- **IPPort** (*int*) – IP Port Number used for TCP and UDP Communications
- **Interface** (*string*) – Defines the network interface on which to listen ('Any', 'LAN', or 'AVLAN')

GVEStatus

Returns:

GVEStatus table

Return type:

dict

Hostname

Returns:

Hostname or IP Address of the GVE Server

Return type:

string

`IPPort`

Returns:

IP Port Number used for TCP and UDP Communications

Return type:

int

`MACAddress`

Returns:

MACAddress of the Processor

Return type:

string

`SendStatus`(*device*, *command*, *value*)

Send Status to GVE Server

External Method that interprets and assembles commands to send to GVE Server. See the SendStatus() table in the appendix for available commands.

Parameters:

- **device** (*string*) – device ID assigned in GVE Server
- **command** (*string*) – friendly name for command sent to GVE Server
- **value** (*string*) – command values to set on GVE Server

`SendSecondaryProcessorStatus`(*device*, *value*)

Send Secondary Processor Status to GVE Server

External Method that sends Secondary Processor Connection Status to GVE Server

Parameters:

- **device** (*extronlib.device.ProcessorDevice*) – Secondary Processor added in GVE
- **value** (*string*) – 'Unknown', 'Connected', 'Disconnected', 'Online' or 'Offline'

property **`DiagnosticSendText`**

`Event:` Triggers when data is sent to the GVE server.

The callback function must accept exactly two parameters, which are the `gveClient` that triggers the event and the string that was sent.

property **`DiagnosticReceiveText`**

`Event:` Triggers when data is received from the GVE server.

The callback function must accept exactly two parameters, which are the `gveClient` that triggers the event and the string that was received.

property **`GVEServerConnected`**

`Event:` Triggers when a connection to the GVE server is established.

The callback function must accept exactly two parameters, which are the `gveClient` that triggers the event and a string (`'Connected'`).

property **`GVEServerDisconnected`**

`Event:` Triggers when a connection to the GVE server is closed.

The callback function must accept exactly two parameters, which are the **`gveClient`** that triggers the event and a string (`'Disconnected'`).

property **`ReceiveGVECommand`**

`Event:` Triggers when a command is received from the GVE server.

The callback function must accept exactly two parameters, which are the **`gveClient`** that triggers the event and a tuple `(deviceID, command, parameter)`. See the ReceiveGVECommand in the appendix for the possible commands.

property **`ReceiveGVERoomEvent`**

`Event:` Triggers when a room event action is received from the GVE server.

The callback function must accept exactly two parameters, which are the **`gveClient`** that triggers the event and a tuple `(deviceID, room, eventName)`. See the ReceiveGVERoomEvent in the appendix for the possible room events.

Examples

Instantiation

```
from gve_interface import gveClient # module import statement

MainProcessor = ProcessorDevice('Main Processor')
GVEServer = gveClient('computer.domain.com', MainProcessor)
```

SendStatus

```
# setting the status of a device with ID 50.
DEVICE_ID = '50'

GVEServer.SendStatus(DEVICE_ID, 'Connection', 'Connected')
GVEServer.SendStatus(DEVICE_ID, 'Power', 'On')
GVEServer.SendStatus(DEVICE_ID, 'Lamp 1 Hours', '803')

# device ID 9999
PROJECTOR_ID = '9999'

GVEServer.SendStatus(PROJECTOR_ID, 'Connection', 'Connected')
GVEServer.SendStatus(PROJECTOR_ID, 'Power', 'Off')

# Using a TLP's Online and Offline events to call SendStatus for a TLP
# with device ID '01'.
TLP_ID = '01'

@event(MainTLP, ['Online', 'Offline'])
def MainTLPStatus(device, state):
    GVEServer.SendStatus(TLP_ID, 'Connection', state)
```

SendSecondaryProcessorStatus

```
# You can use the processor's Online and Offline events to call
# SendSecondaryProcessorStatus
SecondaryProcessor = ProcessorDevice('Secondary Processor')

@event(SecondaryProcessor, ['Online', 'Offline'])
def NewSecondaryProcessorStatus(device, state):
    GVEServer.SendSecondaryProcessorStatus(device, state)
```

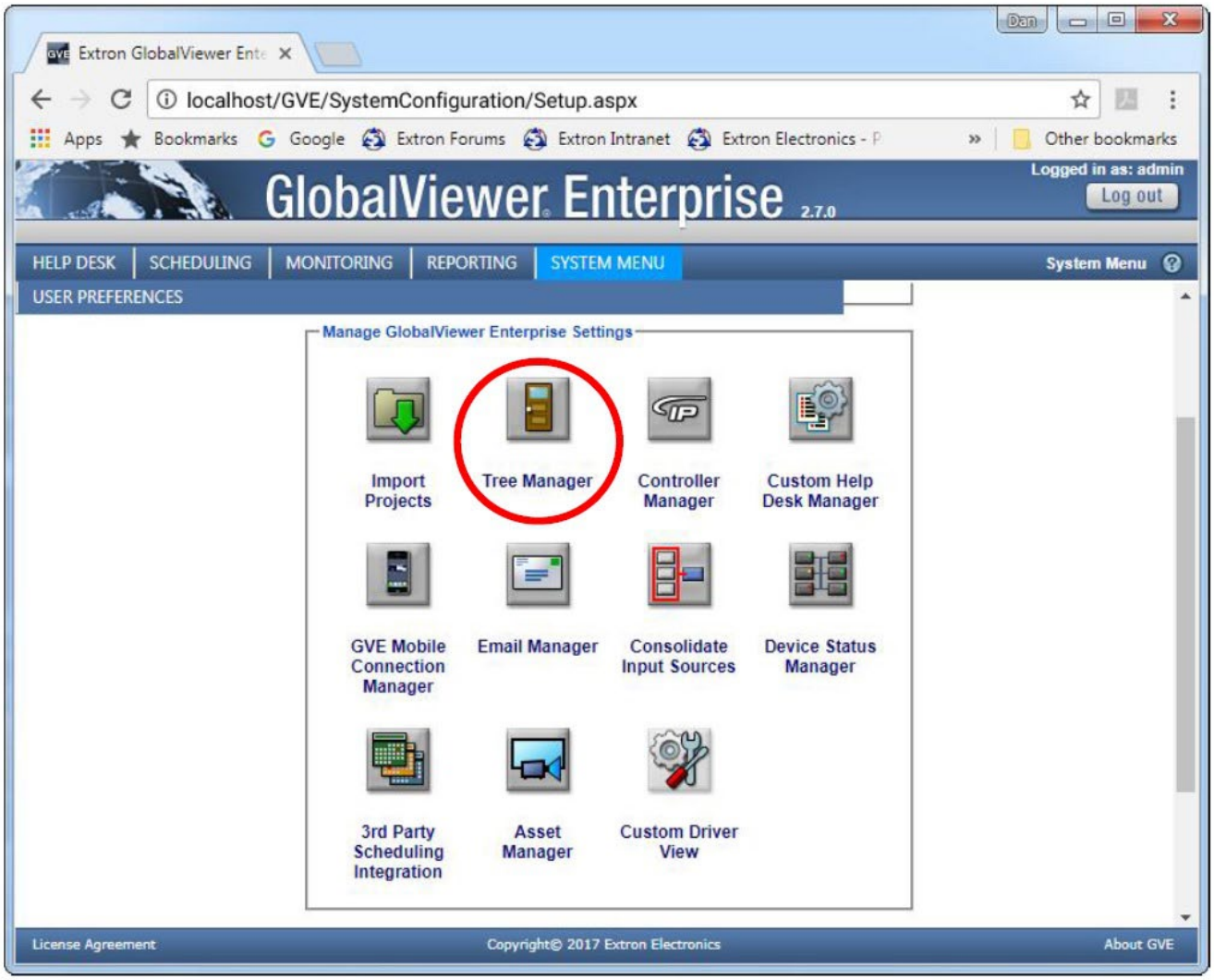
Specifying the interface

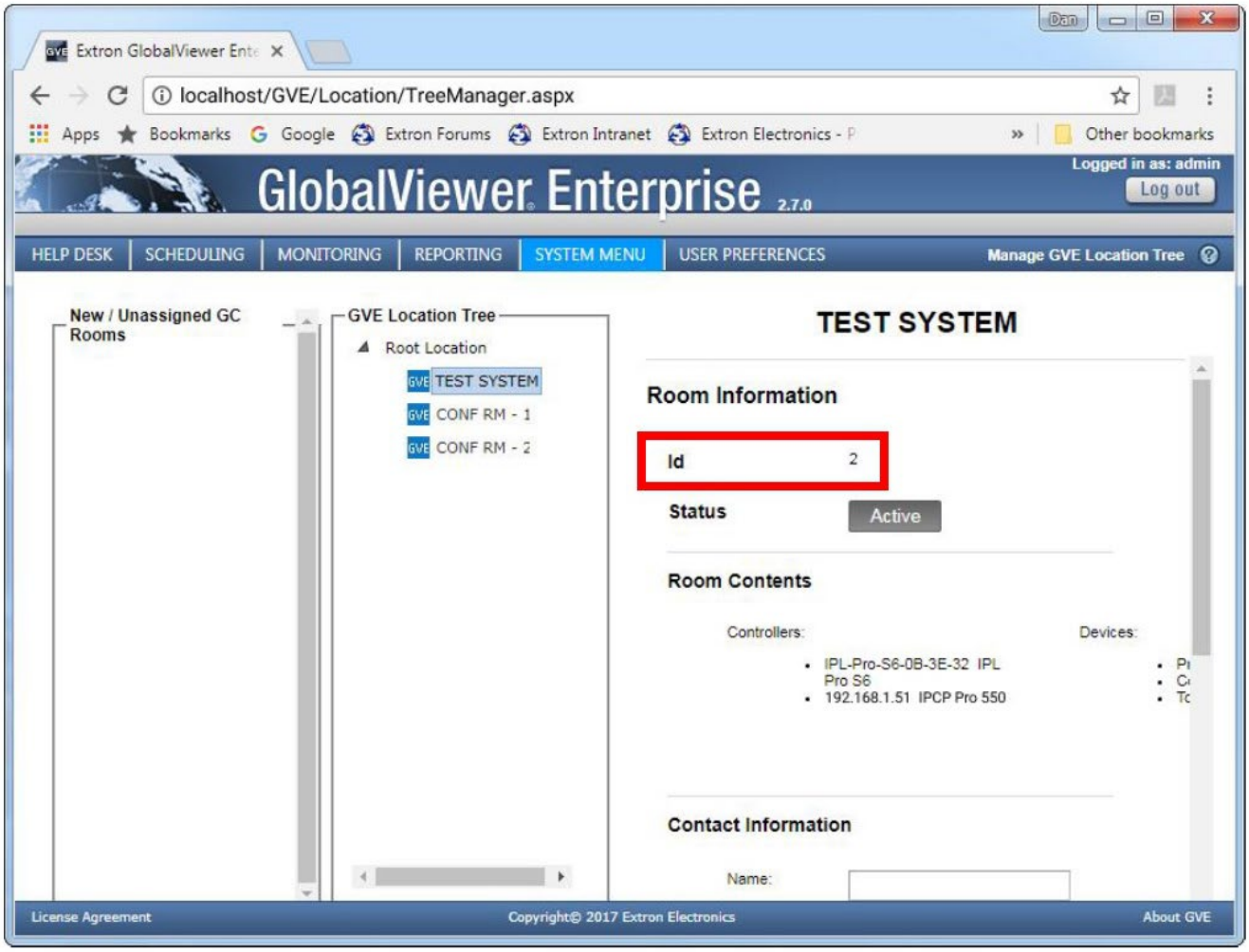
```
from gve_interface import gveClient # module import statement

MainProcessor = ProcessorDevice('Main Processor')

# Using a processor with AVLAN where the device may not be connected to the
# AVLAN at all times.
GVEServer = gveClient('computer.domain.com', MainProcessor, Interface='LAN')
```

Appendix

Finding the Room ID

Room IDs are automatically assigned by GVE when the room is created. Room IDs cannot be edited or manually assigned. Room IDs can be found in the Tree Manager under the System Menu in GVE.


Extron GlobalViewer Ente
localhost/GVE/SystemConfiguration/Setup.aspx
Apps
Bookmarks
Google
Extron Forums
Extron Intranet
Extron Electronics - P
Other bookmarks
GlobalViewer Enterprise 2.7.0
Logged in as: admin
Log out
HELP DESK
SCHEDULING
MONITORING
REPORTING
SYSTEM MENU
System Menu
USER PREFERENCES
Manage GlobalViewer Enterprise Settings
Import Projects
Tree Manager
Controller Manager
Custom Help Desk Manager
GVE Mobile Connection Manager
Email Manager
Consolidate Input Sources
Device Status Manager
3rd Party Scheduling Integration
Asset Manager
Custom Driver View
License Agreement
Copyright© 2017 Extron Electronics
About GVE



`SendStatus()`

Possible values for the `command` and `value` parameters

command	value
‘Power’	‘Unknown’, ‘On’, ‘Off’, ‘Cooling Down’, ‘Warming Up’
‘Source’	<string>
‘Connection’	‘Unknown’, ‘Disconnected’, ‘Connected’, ‘Offline’, ‘Online’
‘Lamp 1 Hours’	<ASCII Digits>
‘Lamp 2 Hours’	<ASCII Digits>
‘Lamp 3 Hours’	<ASCII Digits>
‘Lamp 4 Hours’	<ASCII Digits>
‘Filter Hours’	<ASCII Digits>
‘Device Status’	‘Normal’, ‘Warning’, ‘Error’

`ReceiveGVECommand`

Possible values for the `data` tuple

deviceID	command	parameter
<string>	‘Power’	‘On’, ‘Off’

`ReceiveGVERoomEvent`

Possible values for the `data` tuple

deviceID	room	eventname
<string>	<integer> [1]	‘SystemOn’, ‘SystemOff’

Notes

[1] Numeric value assigned in GVE (integer)

© Copyright 2015-2022, Extron.